Chapter Four: Data Modelling using ER diagrams

4.1 Chapter objectives

By the end of this chapter, learners should:

1. Exhibit understanding of what data modelling is
2. Understand the major
3. Explain the basic components/constructs of ER model
4. Exhibit understanding on how to perform data modelling using ER Model
5. Explain the major problems associated with ER models

4.2 Introduction

Before constructing a database, the first stage is for the system analysts to collect requirements from the end users. These requirements determine what to be represented in the final database and what data should be captured during the database phase. The requirements are represented in a format which the end users can understand in a processing commonly referred to as data modelling

We define can define a data model as a conceptual representation of the data structures that are required by a database. By data structures, we mean the data objects, associations between them as well as the rules which govern operations on these objects. A model will focus on what data is required and how it should be organized rather than what operations will be performed on the data. A model is also independent of any hardware or software constraints which mean that users can use any DBMS to implement the model. Users can also choose any hardware to implement the model

To use a common analogy, the data model is equivalent to an architect's building plans. It serves as a bridge between the concepts that make up real-world events and processes and the physical representation of those concepts in a database.

A data model gets its inputs from the planning and analysis stage. Here the modeler, along with analysts, collects information about the requirements of the database by reviewing existing documentation and interviewing end users.

There are two outputs obtained from a data model. The first is an **Entity Relationship Diagram (ERD)** which represents the data structures in a pictorial form. The diagram is easily learned, it is valuable tool to communicate the model to the end-user. The second component is a **data document** which describes in details the data objects, relationships, and rules required by the

database. The dictionary provides the detail required by the database developer to construct the physical database.

Data modelling is probably the most labor intensive and time consuming part of the development process. You should not build a database without a model just like you shouldn't build a house without blueprints. As a designer, you should ensure that the all data objects required by the database are completely and accurately represented.

4.3 Database design Process

The process of constructing a database always consists of a number of stages. The process is roughly divided in the following five stages

1. Planning and analysis
2. Conceptual design
3. Logical design
4. Physical design
5. Implementation

The data model is one part of the conceptual design process. The other, typically is the functional model. The data model focuses on what data should be stored in the database while the functional model deals with how the data is processed. To put this in the context of the relational database, the data model is used to design the relational tables. The functional model is used to design the queries which will access and perform operations on those tables.

4.4 Importance of data modelling

There a number of benefits associated with data modelling such as

- Since data model uses easily understood notations and natural language, it can be reviewed and verified as correct by the end-users.
- A data model detailed enough to act as a blue print by database developers to build the physical database. The information contained in the data model will be used to define database schema i.e. tables, primary and foreign keys, stored procedures, and triggers.
- A poorly designed database will require more time in the long-term. Without careful planning you may create a database that omits data required to create critical reports, produces results that are incorrect or inconsistent, and is unable to accommodate changes in the user's requirements.

4.5 The Entity-Relationship Model (ER Model)

ER data model is the most common method used to build data models for relational databases. The technique was originally

proposed by Peter in 1976 as a way of unifying the network and relational database views.

4.6 Basic Constructs of ER Modelling

A classical ER model describes data as entities, relationship and attributes

4.6.1 Entities

An entity refers to data object about which information is to be collected e.g. ***employees, projects*** e.t.c. it is analogous to a table in the relational model. We can classify entity as either weak, strong , associative or subtypes

4.6.1.1 Strong entity types

It's an entity that is not existence dependent on some other entity type. One strong characteristic of strong entity is that each entity occurrence is uniquely identifiable using a primary key

4.6.1.2 Weak entity

It's an entity which exist-dependent on some other entity. A strong characteristic of this type of entity is that each entity occurrence cannot be uniquely identified using its attributes and therefore an entity occurrence can only be identifies using a primary key of another entity

4.6.1.3 Special Entity Types

It's an entity used to associate two or more entities in order to reconcile a many-to-many relationship.

4.6.1.4 Subtypes entity

They are used in generalization hierarchies to represent a subset of instances of their parent entity, called the super type, but which have attributes or relationships that apply only to the subset. **4.6.2 Attributes**

Attributes refers to properties that describe an aspect of the entity e.g. a student entity maybe described by properties like RegNo, PostalAddress.

We can classify them as being *simple or composite; single valued or multi-valued; or derived*

4.6.2.1 Simple/atomic vs. composite attributes Simple attribute is one composed of a single component with an independent existence and cannot be divided further into simpler components e.g. RegNo of student entity.

On the other hand, composite attribute is as an attribute composed of multiple components each with independence existence. This type of attributes can be further broken down to

yield smaller components e.g. we can divide a student name to have three other independent attributes; Sname, Lname and Mname.

The decision to model the StudentName as a single attribute or to subdivide it into three parts depends on whether the user view of the data refers to the student name as a single unit or as individual component.

4.6.2.2 Single valued Vs. Multivalued attribute A single valued attribute is one which holds a single value for each entity occurrence of. Majority of attributes fall under this category.

On the hand, a multi valued attribute refers to an attribute that holds multiple values for each entity occurrence. E.g. the email address of students may have multiple values. A multi valued attribute may have a set of numbers with upper and lower limits.

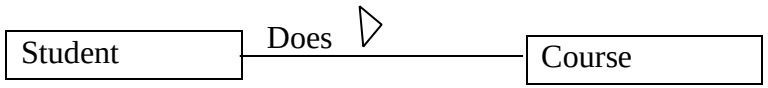
4.6.2.3 Derived vs. stored attribute

A derived attribute represents a value that is generated from the value of related attribute or set attributes not necessarily in the same entity e.g. we can generate age from a stored attribute (dateofbirth).

On the other hand, a stored attribute refers to a value that is physically stored in the database. In the above scenario, date of birth can be termed as a stored attribute.

4.3.1 Relationship

The last component of an ER model is relationship which refers to a set of meaningful association among two or more participating entity. Each relationship has a name that describes what it does.e.g.



We name a relationship using a verb e.g. (e.g. has, manages, takes) or short phrase including a verb (e.g. LeasedBy). After choosing a name for the relationship, a label is always placed beside the name to indicate the direction for the reader to interpretate the relationship name.

A unique identifiable association including one occurrence from each participating entity is usually referred to as **relationship occurrence**. **A degree of relationship** termed as the number. A relationship involving two entities is called binary, while that of three is referred to as ternary and of four quaternary.

Attributes on relationships

It's possible to place attributes on relationships. For example consider the relationship enrols which links the students to units they are doing in a given semester. To track the units the units a student has enrolled for and the date that student enrolled, we link this information with the enrols relationship which has attributes such as datenroled, student_number and course_number.

4.7 Structural constraints

Let's examine the constraints that can be placed on entities that participate on relationship. These constraints should reflect the constraints on the relationship as perceived in the 'real world'. The main constraint on relationship is called multiplicity which refers to the number/range of possible occurrences of an entity that may relate to a single occurrence of another associated entity through a particular relationship.

Multiplicity limits the way entities relate to one another. It's a representation of the business rules established by the user or enterprise. To determine multiplicity, we exam the enterprise rules and constrain using sample data obtained by examining filled in forms, reports or discussion with users. To determine relationship between two users, it is useful to start by asking can instance of A be associated with one or more than one instance of B. Also can instance of B be associated with only one or with more than one instance of A?

There are three types of relationships that can exists between participating entities in a given relationship; one to one (1:1), one-to-many (1: N) or many-to-many (M:N). We examine these constraints using the following integrity constraints

4.7.1 One to one (1:1) relationships

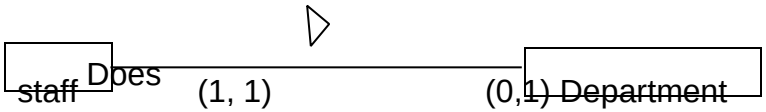
Let's consider the following enterprise constraint ***"A member of staff manages zero or one department and one department can have only one manager".***

We can represent this constraint through a manages relationship that associates staff entity to department Member of staff can manage zero or department and a department is managed by only a member of staff. There is a maximum of one department for each member of staff involved in this and a maximum of one member of staff for each department, we call this relationship as one to one.

To represent that a *member of staff can manage zero or one department we place 0…1* beside the department entity. To

represent that a *department always has one manager we place a*

1…1 beside the staff entity as shown below



4.7.2 One to many (1 :*) relationship

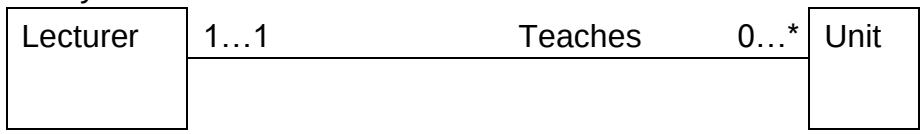
Let's consider the following enterprise constraint/requirement

"*Lectures teaches units"*

The can represented through the *teaches* relationship that associates lecturer and units entities.

Lecturer can teach one or more units. Each unit is taught by one and only one lecturer in a given academic semester.

To represent that a *lecturer can teach one or more units* we place 0…* beside the unit entity. To represent that a *unit can be taught by only one lecturer* we place a 1…1 beside the lecturer entity as shown below



4.7.3 Many to many (* :*) relationships

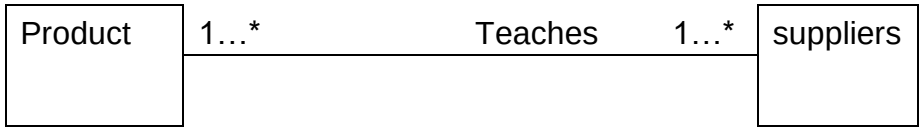
Let's consider the following enterprise constraint/requirement

"*A supplier delivers products"*

The can represented through the *delivers* relationship that associates suppliers and products

A supplier can supply one or many products while a product can be supplied by one or many suppliers

To represent that a *supplier can supplier one or many products* we place 1…* beside the products entity. To represent that a *product can be supplied by one or many suppliers* we place a 1…* beside the product entity as shown below



4.8 Participation and cardinality constraints

Multiplicity contains two separate constraints; cardinality and participation

4.8.1 Cardinality

Describes the maximum number of relationships occurrence for an entity in a relationship for an entity participating in a given relationship. Cardinality of any relationship appears as the ***maximum values for multiplicity range on either side of the relationship***

4.8.2 Participation

Determines whether all or some entity occurrences participate in a given. The participation of entities in relationship appears as the ***minimum values of either side of the relation.***

Types of Participation

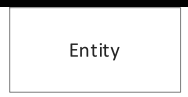
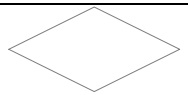
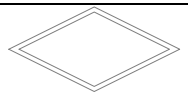
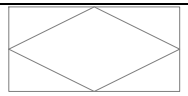
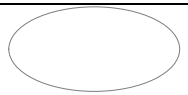
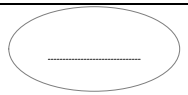
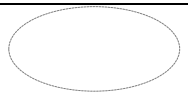
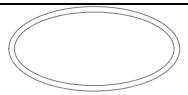
- **Optional** which is represented by minimum value of 0 and occurs if only some entity occurrences are involved in a relationship.
- **Mandatory** which is represented by minimum value of 1 and occurs if all entity occurrences are involved in a relationship

4.9 ERD symbols

There are two commonly used notations when designing ERD models; Chen's or Crow's Foot.

4.9.1 Chen notations

Chen's ERD notation is more detailed way to represent entities and relationships.

Symbol	Meaning
	Entity
	Weak entity
	Relationship
	Identifying weak relationship
	Associative entity
	Attribute
	Key attribute
	Weak attribute
	Derived attribute
	Multivalued attribute
Participations (Mandatory)	
	(0:1)

1 1	(1:1)
N	(0:N)
1 N	(1:N)
M	(0:M)
1 M	(1:M)
Participations (Optional)	
1	(0:1)
1 1	(1:1)
N	(0:N)
N 1	(1:N)
M	(0:M)
1 M	(1:M)

4.9.2 Crow feet Notations

Symbol	Meaning
Entity	Entity with no attributes
	Entities (with attributes field)
	Entity(attributes field with column)
	Entity (attributes field with column and variable number of rows
Cardinality	
	Zero or more
	One or more
	One and only one
	Zero or one
Many to many	
	A zero through many on both sides of a relationship
	A one through many on both sides of a relationship

	A zero through many on one side and a one through many on the other side
One to one relationship	
	A one and only one notation on one side of a relationship and a zero on the other side.
1:1	A one and only one on both sides
Many to one relationship	
	A one through many relationship on one side of relationship and a one and only one on the other side
	A zero through many relationship on one side of relationship and a one and only one on the other side
	A one through many relationship on one side of relationship and a one and a zero or one on the other side
	A zeo through many relationship on one side of relationship and a one and a zero or one on the other side

4.10 ER Modelling

The following are the steps followed during the process of creating a data model for an enterprise. These steps are iterative.

Understand the problem domain i.e. analyse database requirements.

The stage identifying what kind of data is need to be stored in the database, the queries and reports which can be generated from the database. The stage also involves identifying what are the are *important* people, places, physical things, organizations, events and abstract concepts in the organization?

Design a conceptual schema by creating an ER diagram.

The second stage involves a number of activities which include

- Identifying entities. Nouns are candidate to become entities.
- Identifying relationships between entities- verbs are candidate of becoming relationship.
- Draw an ERD without attributes.
- Identify relationship cardinalities i.e. 1:1, 1:N, N:M and participation constraint (Total, Partial)

- Assign attributes to entity types and relationship types. Usually attributes come from nouns, adjectives or adverbs.
- Select primary key for each entity
- Select the Primary Key of relationships using the following rules
  - If 1:1, then the PK of either side entity type may be selected.
  - If 1: N, then the primary key of N-side entity type must be selected.
  - If M: N, then a composite Primary Key consisting of Primary Key of two entity must be used.
  - If ternary, then a composite Primary Key consisting of the Primary Keys of at least two entity. The actual Primary Key selected will vary depending on the cardinality.

Design a logical schema

The logical database design also involves a number of activities as follows

i. Translating the ERD into a relational schema ii. Check normalization (at least 3NF).

iii. Create data dictionaries for the different tables

iv. Do database prototyping & modify the design if necessary.

v. Summarize the design assertion (integrity, security).

Verify the design with users and iterate if necessary.

First, it is important to study the problem domain at hand and analyze database requirements. Write a summary paragraph for the problem domain, considering what data need to be stored and what queries and reports need to be processed. All the information necessary for the identified queries and reports must be included in the summary paragraph. Revise the summary paragraph considering database requirements. Second, from the summary paragraph, find nouns. They are candidates for entity types. To determine whether a noun should be designated as an entity, the following decision rules may be applied.

4.11 Rules for design an ERD model

The following are some of the rules invoked while coming up an ERD for an organization

Rule one

Every entity should be important in its own right within the problem domain. **Rule two**

IF *an object type (noun) has only one property to store* ***THEN it*** *is an attribute of another entity type* ***ELSE***

It is an entity.

Rule three

If *an object type has only one data instance* ***then*** *do not model as an entity type.*

Rule four

If *a relationship needs to have unique identifiers* ***then*** *model it as an entity type*.

The first three rules are used to evaluate object types or nouns, and the fourth rule is used to evaluate relationships or verbs.

Once entities have been assigned, we proceed to identify relationship between those entities. Verbs are useful candidates for relationships.

After an ERD has been built, the following rule can aid in validating the diagram.

Rule 5

If *any verb refers to nouns which are not selected as entity types* ***then*** *do not model it as a relationship.*

If any verb in the ERD fails to follow Rule 5, then consider it again carefully before including it in the diagram. When entities and relationships have been identified, then the cardinality and participation constraints of the relationships can be analyzed.

4.12 Case Study

The following example will illustrate our guidelines for modelling requirements of the problem domain with entity relationship diagrams. Using the summary paragraph of the problem description below, we will progress through the steps of design an ER model for the scenario

A company named xyz has a number of data requirements as listed below. The company keeps track of its employee, departments and projects. After requirements were collected, the database analysts and designers came up with the following description of miniworld (part of the company to be represented in the database).

1. The company is organized in departments. Each department keeps information such as unique name, unique number and particular employee who manages the department. The company also keeps track of the start date when the employee began managing the department. A department may have several locations
2. A department controls a number of projects each which has a unique name, unique number and single locations.
3. The company also stores employee name, ssno, address, salary, gender, birthdate, PostalAddress and TelNo. An employee is assigned to one department but has the option

of working on several projects which are not necessarily controlled by the same department. The company also keeps track of the number of hours per week that an employee works on each project. The company also keeps track of the direct supervisor of each employee.

4. Xyz keeps track of the dependants of each employee for insurance purpose. For this they need fist name of the dependant, his/her gender, birthdate and relationship to the employee.

Required:

Produce an ER model to depict this scenario for XYZ Company.

4.13 Representing Relational Database Schemas

Relational database schemas consists of any number of normalized relations e.g. of a school database application.

Student (Regno, Fname, Lname, Mname, Emailadress, TelNo, Gender)
Staff (SNo, FName, LName, Mname, Emailadress, TelNo, Gender, designation)
Course (CID, CourseName, description)
Student_course (Regno, Cid, semester, academicyear)
Fee_payment (RegNo, Payment_Date, Balance, Trans_Id)
Fee_view (RegNo, date, balance)

The common convection for representing a relation schema is give the name of the relation followed by attributes in parenthesis ( ). A primary key is usually underlined

4.14 Problems with ER Models.

ER model suffers from a common problem called connection trap and usually occurs as a result of misinterpretation of meaning of certain relationships. There are two types of connection traps:

I. **Fan trap:** it exist when a model represents a relationship between entity types but the pathway between certain entity occurrences is ambiguous
II. **Chasm traps:** they exist where a model suggests the existence of a relationship between entity types but the pathway doesn't exist between entity occurrences.

4.15 Chapter summary

4.16 Chapter exercise

Consider the following business rules for a patient appointment system

A doctor can be scheduled for many appointments, but may not have any scheduled at all. Each appointment is scheduled with exactly 1 doctor. A patient can schedule 1 or more appointments. One appointment is scheduled with exactly 1 patient. An appointment must generate exactly 1 bill, a bill is

generated by only 1 appointment. One payment is applied to exactly 1 bill, and 1 bill can be paid off over time by several payments. A bill can be outstanding, having nothing yet paid on it at all.

One patient can make many payments, but a single payment is made by only 1 patient. Some patients are insured by an insurance company. If they are insured, they can only carry insurance with one company. An insurance company can have many patients carry their policies. For patients that carry insurance, the insurance company will make payments; each single payment is made by exactly 1 insurance company.

Required:

Using the crow feet notation and Chen notation, draw the appropriate Entity Relationship model diagram

4.17 Further reading suggestions